

.NET / C# Interview

Study Guide & Cheat Sheet

Mid-Level (2–5 years) • 14-Day Plan

Covers:

OOPS • C# Core • SOLID • Design Patterns • .NET Framework • Threading & Async • Collections • Exception Handling

Table of Contents

1. 14-Day Study Plan

This plan maps directly to the folders you have. Spend ~1.5–2 hrs/day. Do code before reading.

Day	Focus	Folders to Open
1	OOPS basics: class, object, encapsulation, abstraction	OOPS-CSHarp-Code → 1-OOPS Solution, Encapsulation, Abstraction
2	Inheritance, polymorphism, method overloading/overriding	OOPS → Inheritance, Polymorphism, Overloading, Overriding
3	Abstract class vs Interface, sealed classes, static class	OOPS → Abstract vs Interface, Static class, Prevent inheritance
4	Constructors (all types), private constructors, this keyword	OOPS → Constructor types, Private constructor, this
5	Access modifiers, ref/out/params, var vs dynamic, is vs as	OOPS → Access Specifiers, ref vs out, var vs dynamic, is/as
6	Delegates, Multicast, Anonymous, Events vs Delegates	OOPS → Delegates, Multicast, Anonymous, Events
7	Generics, Extension methods, Nullable types, Enums	OOPS → Generics, Extension Methods, Nullable, Enum
8	Collections: Array vs ArrayList, ArrayList vs Hashtable, IEnumerable	OOPS → Array, ArrayList, Hashtable, IEnumerable
9	Exception handling: try/catch/finally, throw vs throw ex, multiple catch	OOPS → Exception Handling, Finally, throw_ex_throw
10	SOLID — SRP, OCP, LSP	SOLID-Code → SOLID, OCP, LSP
11	SOLID — ISP, DIP + Factory, Singleton	SOLID-Code → ISP, DIP, FactoryMethod, SimpleFactory, SingletonDP
12	Threads & Tasks, Async/Await	DotNet-Framework-Code → ThreadTask, AsyncAwait*
13	GC, Dispose vs Finalize, IDisposable, using	DotNet-Framework-Code → GarbageCollector, Dispose-Finalize
14	Assembly, Reflection, Serialization, MVC Filters + mock interview	DotNet-Framework-Code → Assembly, Reflection, Serialization, Filters-MVC

2. OOPS — Object-Oriented Programming in C#

2.1 The 4 Pillars of OOPS

Pillar	One-Line Definition
Encapsulation	Wrap data + behavior in a class; expose only what's needed via properties.
Abstraction	Hide implementation; show only the contract (abstract class / interface).
Inheritance	A class reuses members of another (is-a relationship). C# supports single class inheritance + multiple interfaces.
Polymorphism	Same interface, different behavior — compile-time (overloading) and runtime (overriding).

2.2 Class vs Object

A class is a blueprint; an object is an instance allocated on the heap. Reference types (class) live on the heap and are accessed by reference; value types (struct) live on the stack and are copied by value.

2.3 Access Modifiers

Modifier	Visibility	Default For
public	Anywhere	—
private	Within the same class only	Class members
protected	Same class + derived classes	—
internal	Same assembly	Top-level classes
protected internal	Same assembly OR derived classes	—
private protected	Same assembly AND derived classes (C# 7.2+)	—

2.4 Constructors — All Types

- **Default:** parameterless, generated if no constructor is defined.
- **Parameterized:** initializes with arguments.
- **Copy:** takes another instance and copies its state.
- **Static:** runs once per type before first use; no parameters, no access modifier.
- **Private:** prevents external instantiation — used for Singleton and factory-only construction.

```
public class Logger {
    private static Logger _instance;
```

```
private Logger() { } // private ctor
public static Logger Instance =>
    _instance ??= new Logger(); // singleton accessor
}
```

2.5 Method Overloading vs Overriding vs Hiding

Concept	How	Polymorphism Type
Overloading	Same name, different signatures in same class	Compile-time (static)
Overriding	virtual in base, override in derived — dispatch via vtable	Runtime (dynamic)
Hiding (new)	Derived class hides base member with new keyword	Neither — name reuse

```
class Base { public virtual void Show() => Console.WriteLine("Base"); }
class Derived : Base {
    public override void Show() => Console.WriteLine("Derived"); // overriding
    public new void Hello() { /* hides base */ }
}
```

2.6 Abstract Class vs Interface

Abstract Class	Interface
Can have implementation + abstract members	Contract only (default methods allowed in C# 8+)
Single inheritance	Multiple interfaces can be implemented
Can have constructors, fields, access modifiers	No constructors, no instance fields
Use when classes share state/behavior (is-a)	Use when unrelated classes share a capability (can-do)
Members default: private; explicitly mark abstract/virtual	Members default: public abstract

2.7 Static Class

- Cannot be instantiated; cannot be inherited; all members must be static.
- Loaded once per AppDomain; useful for utilities (Math, Console).
- Static constructor runs once before first member access.

2.8 Sealed / Preventing Inheritance

```
public sealed class FinalClass { /* cannot be inherited */ }
public class Base {
```

```
public sealed override void Show() { } // sealed override blocks further override
}
```

2.9 Quick Concepts (Mid-Level Must-Knows)

Concept	Key Point
this keyword	Refers to current instance; used to disambiguate fields, chain ctors via this(...).
Boxing/Unboxing	Boxing: value type → object (heap allocation, perf hit). Unboxing: cast object back to value type.
ref vs out	Both pass by reference. ref: must be initialized before passing. out: must be assigned inside the method.
params	Variable-length argument list — last parameter, must be 1-D array.
var vs dynamic	var: compile-time inferred, type-checked. dynamic: resolved at runtime, no IntelliSense, can throw <code>RuntimeBinderException</code> .
is vs as	is: returns bool. as: returns null on failure (only reference/nullable types).
Nullable<T>	Value types that can hold null — <code>int?</code> is shorthand for <code>Nullable<int></code> .
Enum	Strongly-typed named constants; default underlying type is <code>int</code> .
readonly vs const	const: compile-time, implicitly static. readonly: runtime, set in declaration or ctor.
string vs StringBuilder	string is immutable — reuse builds new objects. <code>StringBuilder</code> is mutable — use in loops/heavy concatenation.

3. Collections & Generics

3.1 Array vs ArrayList vs List<T>

Feature	Array	ArrayList / List<T>
Size	Fixed	Dynamic
Type-safety	Strongly-typed	ArrayList: object (boxing). List<T>: strongly-typed.
Performance	Fastest indexed access	List<T>: near-array. ArrayList: slower due to boxing.
Recommendation	Fixed-size known at compile time	Prefer List<T>; avoid ArrayList in new code.

3.2 ArrayList vs Hashtable

ArrayList	Hashtable
Index-based: items accessed by integer index	Key-based: items accessed by hash of a key
O(n) search	O(1) average lookup
Stores object only — boxing	Stores object key + object value
Modern equivalent: List<T>	Modern equivalent: Dictionary<TKey,TValue>

3.3 IEnumerable / IEnumerator / IQueryable

- IEnumerable<T> — read-only forward iteration; the foreach contract.
- IEnumerator<T> — actually walks the sequence (Current/MoveNext/Reset).
- IQueryable<T> — defers execution and translates to a query (e.g., SQL via EF). Use for LINQ-to-DB; use IEnumerable for in-memory.

3.4 Generics — When and Why

- Type-safety without boxing/casting performance hit.
- Reusability: write once for any type.
- Constraints: where T : class / struct / new() / SomeBase / ISomeInterface.

```
public class Repository<T> where T : class, IEntity, new() {
    public T Create() => new T();
    public void Save(T entity) { /* ... */ }
}
```

4. Delegates, Events, Lambdas

4.1 Delegate

A type-safe function pointer. Holds a reference to a method with a matching signature.

```
public delegate int Op(int a, int b);
Op add = (a, b) => a + b;
Console.WriteLine(add(3, 4)); // 7
```

4.2 Multicast Delegate

A delegate that points to multiple methods. Combine with += / -=. Each method runs in order; return value is the last one's.

```
Action notify = () => Console.WriteLine("A");
notify += () => Console.WriteLine("B");
notify(); // prints A then B
```

4.3 Anonymous Methods & Lambda

- Anonymous: `delegate(int x) { return x*2; }` — older syntax.
- Lambda: `x => x*2` — modern, concise.
- Both close over (capture) outer variables.

4.4 Built-in Delegates

Type	Signature
Action	Returns void; up to 16 input params
Func<...,TResult>	Returns TResult; last generic arg is return type
Predicate<T>	Func<T,bool> — used for filters

4.5 Events vs Delegates

Delegate	Event
Can be invoked from outside the declaring class	Can only be raised from inside the declaring class
Can be assigned with =	Only += / -= allowed
Used for callbacks, function passing	Used for publish/subscribe (UI, messaging)

```
public class Button {
    public event EventHandler Click;
    protected virtual void OnClick() => Click?.Invoke(this, EventArgs.Empty);
}
```


5. Exception Handling

5.1 try / catch / finally

- Multiple catch blocks allowed — order from most-specific to most-general.
- A try block CAN exist without catch, but only if a finally is present.
- finally always runs (even on return/throw) — use it for cleanup. Don't put return in finally.

```
try {
    /* risky work */
}
catch (FileNotFoundException ex) { /* specific first */ }
catch (IOException ex)          { /* broader I/O */ }
catch (Exception ex)           { /* fallback */ }
finally { /* always cleanup */ }
```

5.2 throw vs throw ex (Critical!)

throw;	throw ex;
Preserves the original stack trace	Resets the stack trace to current line — loses origin info
Use this in catch blocks	Avoid — debugging nightmare
Re-throws as-is	Throws like a new exception

5.3 Custom Exceptions

```
public class OrderNotFoundException : Exception {
    public int OrderId { get; }
    public OrderNotFoundException(int id, string msg) : base(msg) { OrderId = id; }
}
```

6. SOLID Principles

SOLID is a set of 5 design principles to make code maintainable, extensible, and testable.

6.1 S — Single Responsibility Principle (SRP)

A class should have only one reason to change.

```
// BAD: Report does two things
class Report { public string Build() {...} public void Save() {...} }
// GOOD: separate concerns
class Report { public string Build() {...} }
class ReportSaver { public void Save(Report r) {...} }
```

6.2 O — Open/Closed Principle (OCP)

Open for extension, closed for modification — add features by adding new types, not editing existing ones.

```
abstract class Discount { public abstract decimal Apply(decimal p); }
class HolidayDiscount : Discount { public override decimal Apply(decimal p) => p *
0.9m; }
class StudentDiscount : Discount { public override decimal Apply(decimal p) => p *
0.8m; }
// Adding a new discount = new class, no edits to existing code.
```

6.3 L — Liskov Substitution Principle (LSP)

Subtypes must be substitutable for their base type without breaking behavior.

- Classic violation: Square inherits Rectangle and breaks the SetWidth/SetHeight contract.
- Symptom: derived class throws NotImplementedException for a base method — the hierarchy is wrong.

6.4 I — Interface Segregation Principle (ISP)

Clients should not be forced to depend on interfaces they don't use. Many small interfaces beat one fat one.

```
// BAD: forces every printer to implement everything
interface IMachine { void Print(); void Scan(); void Fax(); }
// GOOD: split
interface IPrinter { void Print(); }
interface IScanner { void Scan(); }
interface IFax { void Fax(); }
```

6.5 D — Dependency Inversion Principle (DIP)

High-level modules should depend on abstractions, not concrete classes. Inject dependencies via constructor.

```
public interface ILogger { void Log(string msg); }
public class OrderService {
    private readonly ILogger _logger;
    public OrderService(ILogger logger) => _logger = logger; // inject abstraction
}
```

7. Design Patterns (Most-Asked)

7.1 Singleton

Ensure a class has only one instance and provide global access to it.

```
public sealed class Config {
    private static readonly Lazy<Config> _instance = new(() => new Config());
    public static Config Instance => _instance.Value;
    private Config() { }
}
```

- Use Lazy<T> for thread-safe lazy initialization.
- Mark sealed and use a private constructor.
- Beware: singletons make testing hard — prefer DI-managed singletons in real apps.

7.2 Factory Method

Defines an interface for creating an object, but lets subclasses decide which class to instantiate.

```
abstract class Logger { public abstract void Log(string m); }
class FileLogger : Logger { public override void Log(string m) { /*...*/ } }
class DbLogger : Logger { public override void Log(string m) { /*...*/ } }

abstract class LoggerFactory { public abstract Logger Create(); }
class FileLoggerFactory : LoggerFactory { public override Logger Create() => new FileLogger(); }
```

7.3 Simple Factory

Not a GoF pattern, but very common: a single class returns one of several concrete types based on a parameter.

```
public static class ShapeFactory {
    public static IShape Create(string kind) => kind switch {
        "circle" => new Circle(),
        "square" => new Square(),
        _ => throw new ArgumentException(nameof(kind))
    };
}
```

8. .NET Framework Internals

8.1 Assembly

- A compiled unit of deployment: .dll (library) or .exe (app).
- Contains: IL code, manifest (metadata), type info, resources.
- **Private** — local to one app (in bin folder). **Shared** — installed in GAC (Global Assembly Cache), strong-named, versioned.
- Strong name = name + version + culture + public key — guarantees uniqueness.

8.2 Reflection

Inspect or invoke types/members at runtime via System.Reflection. Use cases: serializers, DI containers, ORMs, plugin loaders.

```
Type t = typeof(MyClass);
foreach (var m in t.GetMethods())
    Console.WriteLine(m.Name);

object instance = Activator.CreateInstance(t);
t.GetMethod("Run")?.Invoke(instance, null);
```

- Performance: reflection is slow — cache MethodInfo / use compiled expressions for hot paths.

8.3 Serialization

Format	Library	When
JSON	System.Text.Json (modern), Newtonsoft.Json	Web APIs, config, logs
XML	XmlSerializer, DataContractSerializer	Legacy/SOAP/interop
Binary	BinaryFormatter (deprecated), MessagePack/Protobuf	Compact/perf-sensitive

- Mark non-serializable fields with [JsonIgnore] / [NonSerialized].
- BinaryFormatter is deprecated due to security; use MessagePack or Protobuf instead.

8.4 Garbage Collector (GC)

CLR's automatic memory manager. Reclaims unreachable objects on the managed heap.

- **Generation 0** — short-lived (most allocations).
- **Generation 1** — survived one GC; buffer between Gen0 and Gen2.
- **Generation 2** — long-lived; full collection is most expensive.
- Large Object Heap (LOH) — objects ≥85KB; collected with Gen2.
- GC.Collect() exists but should almost never be called manually.

8.5 Dispose vs Finalize

Dispose (IDisposable)	Finalize (~Class destructor)
-----------------------	------------------------------

Called explicitly by you (or via using)	Called by GC, non-deterministic
Releases unmanaged resources immediately	Last-chance cleanup; runs on finalizer thread
Fast and predictable	Adds GC overhead; objects with finalizer survive an extra generation
Implement when holding files/sockets/handles	Avoid unless wrapping unmanaged resources
<pre> public class FileWriter : IDisposable { private FileStream? _fs; public void Dispose() { _fs?.Dispose(); GC.SuppressFinalize(this); } ~FileWriter() { _fs?.Dispose(); } // finalizer as safety net } // usage using (var w = new FileWriter()) { /* auto-disposed */ }</pre>	

8.6 Threading vs Tasks

Thread	Task
Low-level OS thread	Higher-level abstraction over a thread (uses ThreadPool)
Heavy: ~1MB stack	Lightweight; reused
Manual lifecycle	Returns a Task<T>; supports continuations, cancellation, await
Use rarely (foreground/specialized)	Default choice for parallel/async work

8.7 async / await

- await frees the calling thread while an I/O-bound operation completes.
- async marks a method as awaitable; the method returns Task / Task<T> / ValueTask.
- Avoid async void (except for event handlers) — exceptions can't be caught by callers.
- CPU-bound? Use Task.Run. I/O-bound? Use the native async API (HttpClient, EF, file I/O).

```

public async Task<string> GetUserAsync(int id) {
    using var http = new HttpClient();
    var resp = await http.GetAsync($"/api/users/{id}"); // non-blocking
    resp.EnsureSuccessStatusCode();
    return await resp.Content.ReadAsStringAsync();
}
```

8.8 ASP.NET MVC Filters

Filter Type	Runs	Use For
Authorization	Before everything	AuthZ checks (e.g., [Authorize])
Resource	After auth, before model binding (Core only)	Caching, short-circuit

Action	Before/after action method	Logging, modify args/result
Result	Before/after action result	Modify response
Exception	On unhandled exception	Centralized error handling

9. Top 50 Quick-Fire Q&A

Q: What is the difference between value type and reference type?

A: Value types (struct, int, enum) live on the stack and are copied by value. Reference types (class, string, array) live on the heap; variables hold a reference.

Q: Why is string immutable?

A: Performance via interning + thread safety. Any 'modification' creates a new string. Use StringBuilder for repeated concatenation.

Q: Difference between == and Equals()?

A: For value types both compare values. For reference types, == compares references by default; Equals() can be overridden for value-equality (string overrides it).

Q: What is IDisposable for?

A: A contract for deterministic cleanup of unmanaged resources. Implement it when your class holds files, sockets, DB connections; consumers call Dispose() — typically via using.

Q: When should you use struct over class?

A: Small (<16 bytes), short-lived, immutable, value semantics, no inheritance needed. Otherwise prefer class.

Q: What is the difference between Task and Thread?

A: Thread is an OS-level execution unit. Task is a higher-level promise of work that runs on the ThreadPool — supports async/await, continuations, and cancellation.

Q: What does async void mean and when is it OK?

A: Used only for event handlers. Don't use it elsewhere — exceptions propagate to the SynchronizationContext and crash the process.

Q: What is the difference between await and Task.Result?

A: .Result blocks the calling thread and can deadlock in UI/ASP.NET classic. await is non-blocking and propagates exceptions cleanly.

Q: How does GC decide what to collect?

A: Anything not reachable from a root (static field, local variable, CPU register, finalizer queue). Reachability is checked per generation.

Q: What is the difference between IEnumerable and IQueryable?

A: IEnumerable iterates in-memory (LINQ-to-Objects). IQueryable builds an expression tree translated by a provider (LINQ-to-SQL/EF) — query runs on the server.

Q: Why prefer composition over inheritance?

A: Composition is more flexible: you can swap behaviors at runtime, avoid fragile base class problems, and keep classes focused (SRP).

Q: What is the Dependency Injection container's job?

A: Build object graphs by resolving dependencies and managing lifetimes (transient/scoped/singleton).

Q: How does the using statement work?

A: Compiler wraps the block in try/finally and calls Dispose() in the finally — guarantees cleanup even on exception.

Q: What is the difference between const and readonly?

A: **const:** compile-time constant, implicitly static, must be a primitive/string. **readonly:** assigned at declaration or in constructor; can be runtime value or reference type.

Q: What is method hiding?

A: Using `new` in a derived method to hide a base method (not override). Calls dispatch by the static (compile-time) type, not runtime type.

Q: Can a struct inherit from a class?

A: No — structs cannot inherit (other than from `System.ValueType`). They CAN implement interfaces.

Q: What is the role of CLR?

A: Common Language Runtime — JIT compiles IL to machine code, manages memory (GC), enforces type safety, handles exceptions, threading.

Q: What is JIT?

A: Just-In-Time compiler — converts IL to native machine code at first method call; subsequent calls run native.

Q: Difference between managed and unmanaged code?

A: Managed runs under the CLR (memory/security/exceptions managed). Unmanaged runs outside CLR (e.g., C/C++ DLLs called via P/Invoke or COM).

Q: What is boxing/unboxing performance impact?

A: Each boxing allocates on the heap and copies the value. In tight loops over object collections you can multiply allocations massively — use generics.

Q: What is the difference between ref and out?

A: Both pass by reference. **ref:** caller MUST initialize before passing. **out:** callee MUST assign before returning.

Q: Why use generics?

A: Type-safety + no boxing + reusability. Compile-time checks catch errors that `ArrayList` wouldn't.

Q: What is covariance / contravariance?

A: Covariance: `IEnumerable<Derived>` assignable to `IEnumerable<Base>` (out T). Contravariance: `Action<Base>` assignable to `Action<Derived>` (in T).

Q: What does sealed do?

A: On a class: prevents inheritance. On a method: prevents further override down the chain.

Q: Can interfaces have default implementations?

A: Yes, since C# 8 — gives you a way to add to interfaces without breaking existing implementers.

Q: Difference between abstract method and virtual method?

A: **abstract:** no body, derived MUST override. **virtual:** has body, derived MAY override.

Q: What is a partial class?

A: A class definition split across files; combined at compile time. Used for designer-generated code (WinForms, EF) and large classes.

Q: What is the difference between throw and throw ex?

A: `throw` preserves the original stack trace; `throw ex` resets it. Always use `throw`.

Q: What is the role of HttpContext?

A: Per-request container holding Request, Response, User, Session, Items — accessible across the request pipeline.

Q: What's the difference between Singleton and Static class?

A: Static class: cannot be instantiated, no state instance, can't implement interfaces. Singleton: one instance, can implement interfaces, can be passed as a dependency.

Q: What is the diamond problem and how does C# avoid it?

A: Multiple inheritance ambiguity. C# allows only single class inheritance; multiple-interface ambiguity requires explicit implementation.

Q: What is the purpose of yield?

A: Lazy enumeration — yield return produces values one at a time, building an iterator state machine. Saves memory for large/infinite sequences.

Q: What is the difference between Func, Action, and Predicate?

A: Action returns void. Func<..., TResult> returns a value (last generic = return type). Predicate<T> is Func<T, bool>.

Q: Can a class implement two interfaces with the same method?

A: Yes — use explicit interface implementation: void IFoo.Run() { ... } void IBar.Run() { ... }.

Q: What is the use of StringBuilder?

A: Mutable string. Each Append modifies the buffer instead of allocating a new string — far faster in loops.

Q: What is reflection's typical use?

A: Frameworks: serializers, DI containers, ORMs, plugin systems, attribute-based discovery (e.g., test runners).

Q: What are extension methods?

A: Static methods that appear as instance methods on the extended type. Declared in a static class with the first parameter prefixed by this.

Q: What is closure?

A: A lambda/anonymous method that captures a local variable from the enclosing scope. The compiler hoists the variable into a generated class to keep it alive.

Q: What is async streaming (IEnumerable<T>)?

A: Like IEnumerable but each MoveNext is awaitable — used for streaming large async datasets (await foreach).

Q: Difference between Lazy<T> and singleton?

A: Lazy<T> defers construction until .Value is accessed; thread-safe by default. Useful inside Singleton implementations.

Q: What is the GC Suppress Finalize call for?

A: Tells the GC to skip the finalizer when Dispose has already done the cleanup — avoids the extra Gen-2 promotion.

Q: What is a memory leak in managed code?

A: Objects unintentionally kept alive: static event handlers, never-removed cache entries, captured variables in long-lived closures.

Q: What is the role of GAC?

A: Global Assembly Cache — machine-wide store of strong-named shared assemblies. Less common in modern .NET (Core/.NET 5+ do not use GAC).

Q: What is the difference between AppDomain and Process?

A: Process: OS-level isolation. AppDomain: managed isolation within a process (legacy .NET Framework — removed in .NET Core+).

Q: What's new in C# 8/9/10/11 worth mentioning?

A: C# 8: nullable reference types, default interface methods, switch expressions. C# 9: records, init-only setters, top-level statements. C# 10: file-scoped namespaces, global usings. C# 11: required members, raw string literals.

Q: Why use records?

A: Immutable, value-equality reference types — perfect for DTOs and data transfer. Concise syntax: `public record Person(string Name, int Age);`

Q: What is dependency injection lifetimes?

A: Transient: new instance every resolution. Scoped: one per request. Singleton: one for the app lifetime.

Q: How do you handle exceptions in async code?

A: `await` unwraps `AggregateException` to the original exception. Wrap `await` calls in `try/catch` normally; for `Task.WhenAll()`, iterate `Task.Exception.InnerExceptions`.

Q: What's the difference between `Parallel.ForEach` and `async/await`?

A: `Parallel.ForEach`: CPU-bound work across threads. `async/await`: I/O-bound work without blocking threads. Don't use `Parallel` for I/O — it just wastes threads.

Q: What is unit of work + repository pattern?

A: Repository abstracts data access (one per aggregate). Unit of Work coordinates a set of repositories under a single transaction (commit/rollback).

10. Interview Strategy & Tips

10.1 The STAR-style answer for behavioural questions

1. Situation — set the scene briefly.
2. Task — what you needed to do.
3. Action — what YOU specifically did.
4. Result — outcome with a number, if possible.

10.2 How to answer technical questions

- State the definition in one line.
- Give a real-world analogy or use case.
- Show a small code snippet (mentally).
- Mention a trade-off or pitfall — that's what separates mid from junior.

10.3 Red-Flag Phrases to Avoid

- "I don't know" → say "I haven't worked with that, but my understanding is ..."
- Memorized definitions without examples — interviewers spot it instantly.
- Vague answers like "it's for performance" — quantify or contextualize.

10.4 Mid-Level Differentiators

- Always discuss trade-offs (memory vs speed, simplicity vs extensibility).
- Bring up testing implications: 'this design makes mocking easier because...'
- Mention SOLID/DI when discussing design choices.
- Know the difference between IL/JIT/CLR/CIL — it shows depth.

10.5 Day-Of Checklist

5. Sleep > 7 hrs; arrive (or log in) 10 min early.
6. Have your projects open — be ready to walk through one in 60 seconds.
7. Prepare 2 questions for the interviewer (about the team, the codebase).
8. Re-read your top-10 weakest topics from this guide.
9. Breathe. Pause before answering — silence is fine.

Good luck, Feroze!

This guide is a complement to your existing folders, not a replacement. Run the code examples in OOPS-CSharp-Code, SOLID-Code, and DotNet-Framework-Code as you go through each section — fingers-on-the-keyboard practice beats any cheat sheet.

Three high-leverage moves for the next 14 days:

10. Speak your answers out loud — silent rehearsal hides gaps.
11. Build one tiny project that uses 5+ topics together (e.g., a console app with DI, async, generics, custom exceptions, JSON serialization).
12. On Day 14, do a full mock interview against the Top 50 Q&A above, timing yourself to 90 seconds per answer.